Predicción de cancelaciones de reservas hoteleras con aprendizaje de máquina

Juan Camilo Morales Villada, Juan Sebastián Andraus López, Jose Miguel Fajardo Fernández
Modelos y Simulación de Sistemas II · Universidad de Antioquia, Medellín
Repositorio: github.com/Moraa0711/ModelosII-Proyecto-HotelBooking

***Resumen*—Se presenta un sistema de aprendizaje supervisado para predecir la cancelación de reservas hoteleras a partir del dataset Hotel Booking Demand, con 119.390 reservas de dos hoteles de Portugal. Tras la limpieza de los datos y la exclusión de las variables con fuga de información, se compararon seis modelos (regresión logística, KNN, Random Forest, XGBoost, perceptrón multicapa y SVM) mediante validación cruzada estratificada y agrupada, que evita que reservas idénticas queden a la vez en entrenamiento y prueba. XGBoost obtuvo el mejor desempeño, con un F1 de 0,820 y un AUC-ROC de 0,943 en el conjunto de prueba, seguido muy de cerca por Random Forest. La reducción de dimensión con PCA a 25 componentes (72,2 % menos variables) redujo el F1 en cerca de 0,03, mientras que UMAP produjo una pérdida mucho mayor.**

***Palabras clave*—aprendizaje supervisado, cancelación de reservas, clasificación binaria, XGBoost, reducción de dimensión.**

I. INTRODUCCIÓN

El análisis de datos relacionados con las reservas hoteleras permite identificar patrones asociados con el comportamiento de los clientes y las condiciones que pueden influir en la cancelación de una reserva. En este contexto, el aprendizaje automático ofrece herramientas útiles para construir modelos predictivos capaces de clasificar reservas a partir de variables como el tiempo de anticipación, el tipo de hotel, la duración de la estadía, el segmento de mercado, el tipo de depósito, la tarifa diaria y las características del cliente.

Este proyecto se enfoca en aplicar y evaluar técnicas de clasificación supervisada sobre el conjunto de datos Hotel Booking Demand, propuesto por Antonio, de Almeida y Nunes [1], con el objetivo de predecir si una reserva hotelera será cancelada o no. La Sección II describe el problema y la base de datos; la Sección III resume los trabajos previos; la Sección IV presenta la configuración experimental y los resultados de seis modelos; la Sección V evalúa técnicas de reducción de dimensión, y la Sección VI presenta la discusión y las conclusiones.

II. DESCRIPCIÓN DEL PROBLEMA

Las cancelaciones afectan la planificación de la ocupación, la disponibilidad de habitaciones y la gestión de los recursos de un hotel. El problema consiste en clasificar cada reserva en una de dos clases: no cancelada o cancelada. Una solución basada en Machine Learning resulta útil porque permite encontrar de manera automática los patrones presentes en miles de reservas y estimar el riesgo de cancelación de una reserva nueva, información con la que el hotel podría, por ejemplo, solicitar un depósito o confirmar la reserva con el cliente. El propósito del proyecto es principalmente académico: comparar diferentes modelos de clasificación, analizar su desempeño y estudiar el efecto de las etapas de preprocesamiento y de reducción de dimensión.

El dataset contiene 119.390 reservas de dos hoteles de Portugal, un City Hotel y un Resort Hotel, con fechas de llegada entre julio de 2015 y agosto de 2017 [1]. La versión utilizada fue obtenida de Kaggle [2] y tiene 36 variables: las 32 del conjunto original y cuatro columnas (name, email, phone-number y credit_card) creadas de forma artificial por quien publicó esta versión, ya que en los datos originales se eliminó toda la información que permitía identificar a los clientes. La variable objetivo es is_canceled. La Tabla I resume las variables del dataset.

TABLA I
RESUMEN DE LAS VARIABLES DEL DATASET

Tipo de variable	**Nombre**	**Descripción**
Categórica nominal	hotel	Tipo de hotel: *City Hotel* o *Resort Hotel*.
Binaria (objetivo)	is_canceled	Indica si la reserva fue cancelada.
Numérica entera	lead_time	Días entre la reserva y la fecha de llegada.
Temporales	arrival_date_year, arrival_date_month, arrival_date_week_number, arrival_date_day_of_month	Año, mes, semana y día de la fecha de llegada.
Numéricas enteras	stays_in_weekend_nights, stays_in_week_nights	Noches de fin de semana y entre semana.
Numéricas enteras	adults, children, babies	Número de adultos, niños y bebés.
Categórica nominal	meal	Plan de alimentación.
Categórica nominal	country	País de origen del huésped.
Categóricas nominales	market_segment, distribution_channel	Segmento de mercado y canal de la reserva.
Binaria	is_repeated_guest	Indica si el cliente es recurrente.

Tipo de variable	Nombre	Descripción
Numéricas enteras	previous_cancellations, previous_bookings_not_canceled	Reservas previas canceladas y no canceladas del cliente.
Categóricas nominales	reserved_room_type, assigned_room_type	Habitación reservada y habitación asignada.
Numéricas enteras	booking_changes, days_in_waiting_list, required_car_parking_spaces, total_of_special_requests	Cambios, días en lista de espera, parqueaderos y solicitudes especiales.
Categórica nominal	deposit_type	Tipo de depósito.
Numérica continua	adr	Tarifa diaria promedio (*Average Daily Rate*).
Categórica nominal	customer_type	Tipo de cliente.
Identificadores	agent, company	Agente o compañía asociados, si existen.
Categórica / fecha	reservation_status, reservation_status_date	Estado final (*Canceled*, *Check-Out*, *No-Show*) y su fecha.
Texto (sintéticas)	name, email, phone-number, credit_card	Datos personales generados artificialmente.

En cuanto a la variable objetivo, 75.166 reservas (62,96 %) no fueron canceladas y 44.224 (37,04 %) sí lo fueron, por lo que el conjunto presenta un desbalance moderado. En el City Hotel se canceló el 41,73 % de las reservas y en el Resort Hotel el 27,76 % (Fig. 1a). Además, la tasa de cancelación aumenta con la anticipación de la reserva, desde 9,6 % en las reservas hechas con una semana o menos hasta 67,7 % en las realizadas con más de un año (Fig. 1b); lead_time es la variable numérica con mayor correlación con is_canceled (0,29).

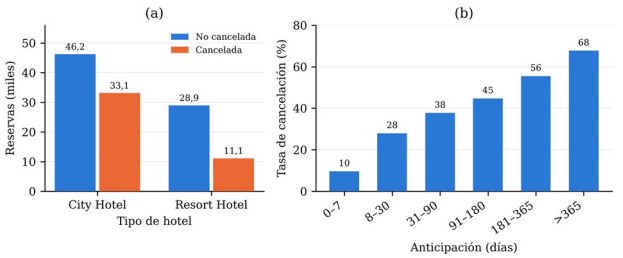


Fig. 1. (a) Número de reservas canceladas y no canceladas por tipo de hotel. (b) Tasa de cancelación según el tiempo de anticipación de la reserva (lead_time).

El conjunto presenta valores faltantes en cuatro variables, que se muestran en la Tabla II junto con su tratamiento. En agent y company la ausencia de valor no es un error de registro, sino que indica que la reserva no se hizo por medio de un agente o de una compañía; además, ambas son identificadores con 333 y 352 valores distintos, por lo que se reemplazaron por dos indicadores binarios. Algunas variables categóricas contienen también la categoría Undefined: en meal (1.169 registros) se agrupó con SC, ya que ambas indican que la reserva no incluye plan de alimentación [1], y en market_segment (2) y distribution_channel (5) se eliminaron esos registros.

TABLA II
VALORES FALTANTES Y ESTRATEGIA DE TRATAMIENTO

Variable	Faltantes	%	Tratamiento
children	4	0,003	Imputación con la moda (0)
country	488	0,41	Categoría adicional “Desconocido”
agent	16.340	13,69	Indicador binario: con o sin agente
company	112.593	94,31	Indicador binario: con o sin compañía

También se eliminaron 180 reservas sin huéspedes, una con tarifa diaria negativa (−6,38) y una con un adr de 5.400, muy superior al resto (el 75 % de las reservas tiene un adr menor o igual a 126). Tras la limpieza quedaron 119.203 reservas. En esta base, 41.807 reservas (35,1 %) son idénticas a al menos otra en todas las variables de entrada; corresponden principalmente a reservas de grupos y de agencias, con grupos de hasta 180 reservas iguales, y el 57,3 % de ellas fue cancelado, frente al 26,2 % en las filas únicas. Como eliminarlas reduciría la tasa de cancelación de 37,1 % a 27,6 % y cambiaría el problema, se conservaron, y las particiones se hicieron de forma agrupada (Sección IV-A).

Algunas variables no se usaron como entrada de los modelos. Las cuatro columnas sintéticas no aportan información real. reservation_status coincide por completo con la variable objetivo (todas las reservas con estado Check-Out tienen is_canceled igual a 0, y todas las Canceled o No-Show, igual a 1), por lo que mantenerla, junto con reservation_status_date, generaría una fuga de información (data leakage). assigned_room_type también se excluyó, ya que se conoce después de realizada la reserva [4]: las reservas con habitación asignada distinta a la reservada se cancelan solo el 5,38 % de las veces, frente a 41,56 % en las demás. La variable country ha sido señalada como posible fuga [3], [4] (Portugal es el 62,32 % de las reservas canceladas y el 28,19 % de las no canceladas); se conservó y su efecto se evaluó en la Sección IV-B.

Las 28 variables de entrada resultantes se codificaron así: *(i)* las categóricas nominales (hotel, meal, market_segment, distribution_channel, reserved_room_type, deposit_type y customer_type) con One-Hot Encoding; *(ii)* country también con One-Hot, agrupando en una sola categoría los países con menos de 100 reservas en el conjunto de entrenamiento; *(iii)* las numéricas y temporales con estandarización (media 0 y desviación 1), después de convertir arrival_date_month a su número de mes; y *(iv)* las

binarias (is_repeated_guest y los indicadores de agente y compañía) sin cambios. Con esto se obtuvieron 90 variables codificadas.

Dado que cada reserva tiene una etiqueta conocida y la variable objetivo tiene dos clases, el problema se abordó con un paradigma de aprendizaje supervisado de clasificación binaria.

III. ESTADO DEL ARTE

La predicción de cancelaciones con este conjunto de datos ha sido abordada en diferentes trabajos. En todos ellos el problema se plantea como una tarea de aprendizaje supervisado de clasificación binaria con is_canceled como variable objetivo, y en la mayoría se trabaja cada hotel por separado, identificando el Resort Hotel como H1 y el City Hotel como H2.

Adil et al. [3] compararon regresión logística, árboles de decisión, AdaBoost, Gradient Boosting y Random Forest, y evaluaron el efecto de dos técnicas de balanceo de clases, SMOTE y SMOTE-ENN. Antes del entrenamiento eliminaron variables como company, agent, country y reservation_status. Los hiperparámetros de Random Forest se ajustaron mediante búsqueda en malla con validación cruzada de 10 particiones. El mejor desempeño lo obtuvo Random Forest con SMOTE-ENN, con una exactitud superior al 95 % y una media geométrica (G-mean) de 95 % en H1 y 96,3 % en H2. Esta métrica combina la sensibilidad y la especificidad como se muestra en (1), donde TP y TN son los verdaderos positivos y negativos, y FP y FN los falsos positivos y negativos.

$$\text{G-mean} = \sqrt{\frac{TP}{TP+FN} \cdot \frac{TN}{TN+FP}} \tag{1}$$

Yang y Miao [4] propusieron una red neuronal basada en árboles (TNN), en la que la importancia de las variables obtenida con un modelo de árboles se utiliza para inicializar los pesos de una red neuronal de propagación hacia adelante. A diferencia del trabajo anterior, dividieron los datos de forma temporal, usando el 75 % de las reservas más antiguas para entrenamiento y el 25 % restante para prueba, con validación cruzada para series de tiempo dentro del conjunto de entrenamiento. Entre los modelos de árboles evaluados (Gradient Boosting, XGBoost, Random Forest y Extra Trees), el mejor AUC-ROC en prueba fue de 0,7718 en H1 y 0,7837 en H2, mientras que la TNN alcanzó 0,8554 y 0,8370, respectivamente. Además del AUC-ROC, los autores reportaron el área bajo la curva de precisión-sensibilidad (AUC-PR), la exactitud balanceada (BA) y el coeficiente de correlación de Matthews (MCC), definidos en (2), (3) y (4), donde P es la precisión y R la sensibilidad.

$$\text{AUC-PR} = \int_0^1 P(R)\, dR \tag{2}$$

$$\text{BA} = \frac{1}{2}\left(\frac{TP}{TP+FN} + \frac{TN}{TN+FP}\right) \tag{3}$$

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}} \tag{4}$$

Arasadina et al. [5] compararon Random Forest y XGBoost utilizando un flujo de preprocesamiento diseñado para evitar fugas de información, en el que la codificación y el tratamiento de valores atípicos se ajustan únicamente con los datos de entrenamiento. Usaron una partición estratificada de 70 %, 15 % y 15 % para entrenamiento, validación y prueba, búsqueda en malla con validación cruzada de 3 particiones y la prueba de McNemar para comparar los modelos. Random Forest obtuvo el mejor desempeño general, con un AUC-ROC de 0,9394, una exactitud de 0,8749 y un F1 de 0,8219, mientras que XGBoost alcanzó la mayor sensibilidad (0,8345).

Finalmente, Silvestre, Antonio y Carrasco [6] trabajaron con 670.343 reservas de cuatro hoteles portugueses con la misma estructura de este dataset. Usando XGBoost, entrenaron un modelo con datos anteriores a la pandemia de COVID-19, con una partición estratificada de 67 % para entrenamiento y 33 % para validación, y lo evaluaron con reservas del periodo de pandemia, obteniendo valores de AUC entre 0,70 y 0,93. Con una ventana deslizante de 24 meses de entrenamiento y 9 meses de prueba, el AUC se ubicó entre 0,85 y 0,99, lo que muestra que los cambios en el comportamiento de los clientes afectan el desempeño de los modelos.

En los trabajos revisados, los ensambles de árboles, en especial Random Forest y XGBoost, obtienen los mejores resultados entre los modelos tradicionales. Sin embargo, el desempeño reportado depende en gran medida de la metodología de validación y del tratamiento de las variables con fuga de información: Random Forest alcanza un AUC-ROC de 0,9394 con una partición estratificada aleatoria [5], mientras que con una partición temporal el mejor modelo de árboles obtiene entre 0,77 y 0,78 [4].

IV. ENTRENAMIENTO Y EVALUACIÓN DE LOS MODELOS

A. Configuración experimental

Del total de 119.203 reservas, el 20 % (23.840) se separó como conjunto de prueba y el 80 % restante (95.363) se usó para entrenar y validar los modelos. La partición se hizo con StratifiedGroupKFold de scikit-learn, de forma estratificada (37,1 % de reservas canceladas en ambos conjuntos) y agrupada: cada grupo de reservas idénticas quedó completo en un solo conjunto, de modo que ninguna reserva de prueba tiene una copia exacta en entrenamiento.

Los hiperparámetros se escogieron mediante búsqueda en malla con validación cruzada de cinco particiones, también estratificada y agrupada, sobre el conjunto de entrenamiento. Para cada combinación se reporta la media de los cinco pliegues con su intervalo de confianza del 95 % (IC95), calculado con la distribución t de Student con cuatro grados de libertad, tanto en entrenamiento como en validación. La codificación y la estandarización se ajustaron dentro de cada pliegue solo con los datos de

entrenamiento de ese pliegue. Con la mejor combinación, cada modelo se reentrenó con todos sus datos de ajuste y se evaluó una sola vez en el conjunto de prueba.

No se usaron técnicas de sobremuestreo ni de submuestreo. Para compensar el desbalance se asignó un peso mayor a la clase minoritaria en los modelos que lo permiten: class_weight balanceado en regresión logística, Random Forest y SVM, y scale_pos_weight igual a 1,70 (razón entre reservas no canceladas y canceladas) en XGBoost. KNN y el perceptrón multicapa de scikit-learn no admiten pesos por clase y se entrenaron sin ellos. Por otra parte, el costo computacional de KNN (en la predicción) y de la SVM con kernel RBF (en el entrenamiento) crece aproximadamente con el cuadrado del número de muestras, por lo que estos dos modelos se ajustaron con una muestra estratificada y agrupada de 15.894 reservas del conjunto de entrenamiento. Todos los modelos se evaluaron con el mismo conjunto de prueba completo.

Se compararon seis modelos que cubren los tipos requeridos: regresión logística (paramétrico), KNN (no paramétrico), Random Forest y XGBoost (ensambles de árboles), un perceptrón multicapa (red neuronal) y una SVM con kernel RBF. La Tabla III presenta los hiperparámetros evaluados y la malla de valores de cada modelo.

La métrica principal fue el F1 de la clase cancelada, con la que se escogió la mejor combinación de hiperparámetros, ya que la exactitud no es adecuada con clases desbalanceadas: un clasificador que siempre predice “no cancelada” obtiene una exactitud de 0,629 y un F1 de 0. También se reportan el AUC-ROC, que mide la capacidad de separar las clases sin depender del umbral y permite comparar con los trabajos de la Sección III; la sensibilidad, que indica qué proporción de las cancelaciones se detecta; la precisión, que indica qué proporción de las alertas es correcta, y la exactitud como referencia.

TABLA III
HIPERPARÁMETROS Y MALLA DE VALORES EVALUADOS

Modelo	**Hiperparámetros y malla**
Regresión logística	C ∈ {0,001; 0,01; 0,1; 1; 10; 100}
KNN*	k ∈ {1; 5; 11; 21; 41; 81}; pesos ∈ {uniformes, por distancia}
Random Forest	Profundidad máxima ∈ {5; 10; 20; sin límite}; mínimo de muestras por hoja ∈ {1; 5}; 200 árboles
XGBoost	Profundidad máxima ∈ {3; 6; 9}; tasa de aprendizaje ∈ {0,03; 0,1; 0,3}; 400 árboles
Perceptrón multicapa	Capas ocultas ∈ {(32); (64); (128, 64)}; α (L2) ∈ {0,0001; 0,01; 1}; parada temprana
SVM (RBF)*	C ∈ {0,1; 1; 10}; γ ∈ {0,003; 0,03; 0,3}

Ajustados con la muestra de 15.894 reservas

B. Resultados

La Fig. 2 muestra el efecto de los hiperparámetros sobre el F1 de validación de cada modelo. En la regresión logística (Fig. 2a) el desempeño se estabiliza desde C = 0,1 y las curvas de entrenamiento y validación casi coinciden, lo que indica que el modelo está limitado por su forma lineal y no por sobreajuste. KNN (Fig. 2b) presenta el caso contrario: el F1 de entrenamiento es cercano a 1, porque con pesos por distancia cada reserva se reconoce a sí misma, mientras que en validación no supera 0,68. En Random Forest (Fig. 2c) y XGBoost (Fig. 2d) el F1 de validación aumenta con la profundidad de los árboles hasta estabilizarse alrededor de 0,81, aunque la brecha con el entrenamiento también crece. En el perceptrón multicapa (Fig. 2e) una regularización fuerte (α = 1) reduce el desempeño, y en la SVM (Fig. 2f) un γ grande (0,3) produce una frontera demasiado irregular que generaliza mal.

La Tabla IV resume el desempeño de la mejor combinación de cada modelo. XGBoost obtuvo el mayor F1 de validación (0,811 ± 0,011) y de prueba (0,820), seguido muy de cerca por Random Forest (0,809 ± 0,012 y 0,818); como sus intervalos de confianza se traslapan, la diferencia entre ambos no es significativa. Ambos alcanzaron un AUC-ROC de prueba de 0,94 (Fig. 3). Les siguen el perceptrón multicapa (F1 de prueba de 0,793), la SVM (0,768), la regresión logística (0,746) y KNN (0,699). En todos los modelos los valores de prueba son cercanos a los de validación, lo que indica que la validación cruzada estimó bien el desempeño sobre datos no vistos.

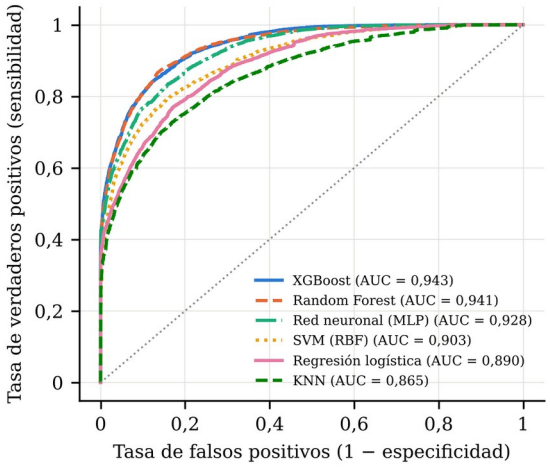


Fig. 3. Curvas ROC de los seis modelos en el conjunto de prueba.

En el conjunto de prueba, XGBoost detectó 7.393 de las 8.839 reservas canceladas (sensibilidad de 0,836), y de las 9.188 reservas que señaló como cancelaciones, 7.393 lo fueron realmente (precisión de 0,805). Random Forest tuvo una precisión mayor (0,829) a cambio de una sensibilidad menor (0,807).

Por último, se evaluó el efecto de la variable country. Al retirarla, el F1 de validación de XGBoost bajó de 0,811 ± 0,011 a 0,756 ± 0,012 y el de Random Forest de 0,809 ± 0,012 a 0,751 ± 0,006; el AUC-ROC pasó de 0,938 a 0,893

y de 0,936 a 0,887, respectivamente. country es, por tanto, una de las variables más informativas, aunque trabajos previos la excluyeron por considerarla una posible fuga [3], [4]. Se mantuvo en el modelo final y esta limitación se discute en la Sección VI.

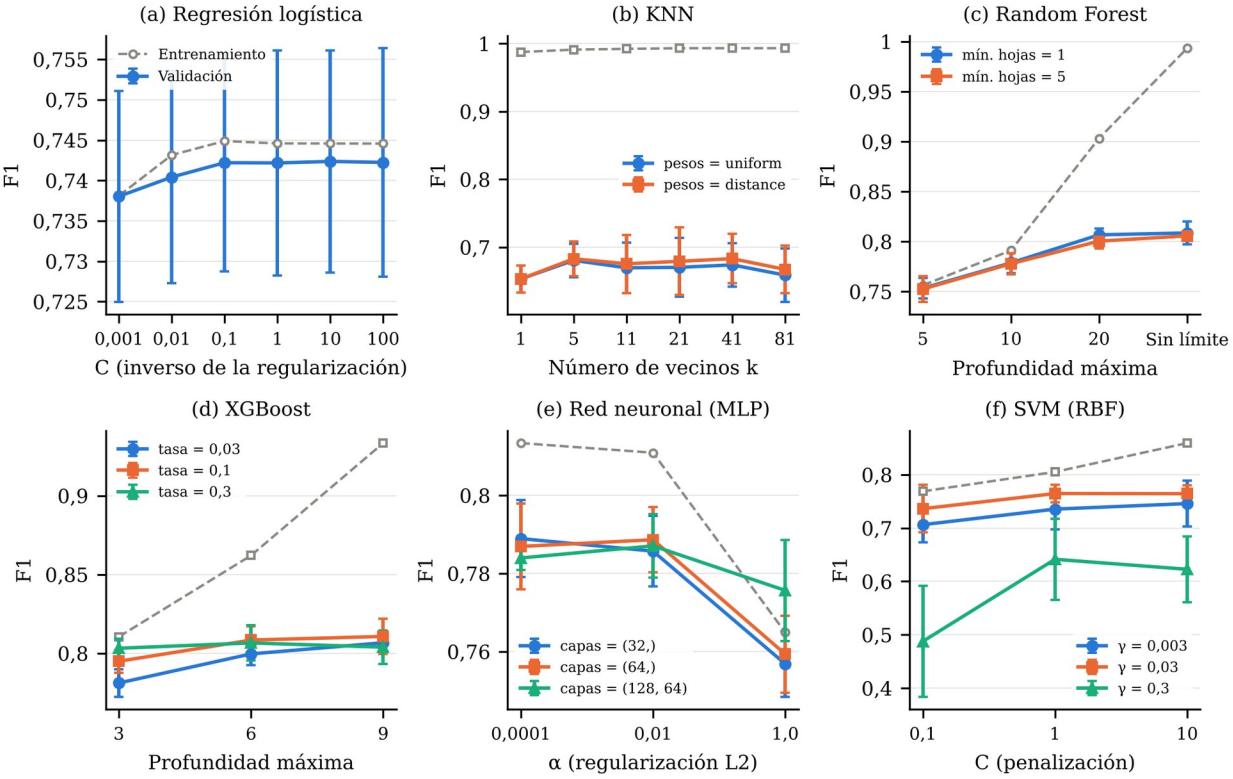


Fig. 2. F1 de validación (media e IC95 de los cinco pliegues) según los hiperparámetros de cada modelo. La línea gris punteada es el F1 de entrenamiento de la serie que contiene la mejor combinación; cuanto más se separa de la validación, mayor es el sobreajuste.

TABLA IV
DESEMPEÑO DE LA MEJOR COMBINACIÓN DE HIPERPARÁMETROS DE CADA MODELO

Modelo	**Mejores hiperparámetros**	**F1 entren.**	**F1 valid.**	**F1 prueba**	**AUC valid.**	**AUC prueba**	**Sensib. prueba**	**Precisión prueba**
XGBoost	Profundidad 9; tasa 0,1	0,933 ± 0,001	0,811 ± 0,011	0,820	0,938 ± 0,004	0,943	0,836	0,805
Random Forest	Sin límite de profundidad; mín. hoja 1	0,993 ± 0,001	0,809 ± 0,012	0,818	0,936 ± 0,003	0,941	0,807	0,829
Perceptrón multicapa	Capa oculta (32); α = 0,0001	0,813 ± 0,006	0,789 ± 0,010	0,793	0,927 ± 0,003	0,928	0,772	0,816
SVM (RBF)*	C = 1; γ = 0,03	0,805 ± 0,001	0,765 ± 0,017	0,768	0,897 ± 0,010	0,903	0,777	0,760
Regresión logística	C = 10	0,745 ± 0,003	0,742 ± 0,014	0,746	0,889 ± 0,007	0,890	0,767	0,726
KNN*	k = 41; pesos por distancia	0,993 ± 0,001	0,683 ± 0,037	0,699	0,852 ± 0,022	0,865	0,628	0,789

Entrenamiento y validación: media ± IC95 de los cinco pliegues. Prueba: evaluación única en 23.840 reservas. *Ajustados con la muestra de 15.894 reservas.

V. REDUCCIÓN DE DIMENSIÓN

A. Análisis individual de variables

Para cada una de las 28 variables de entrada se calculó, con el conjunto de entrenamiento, la información mutua con is_canceled; además, la correlación y el índice de discriminación de Fisher, $J = (\mu_1 - \mu_0)^2 / (\sigma_1^2 + \sigma_0^2)$, para las variables numéricas y binarias, y la V de Cramér para las categóricas. La Fig. 4 muestra la información mutua de cada variable. Las más informativas fueron deposit_type (0,131), lead_time (0,083), adr (0,082) y country (0,067),

mientras que children, babies y meal prácticamente no aportan información por sí solas. En cuanto a la redundancia, el único par con una asociación mayor a 0,8 fue arrival_date_month con arrival_date_week_number (r = 0,995).

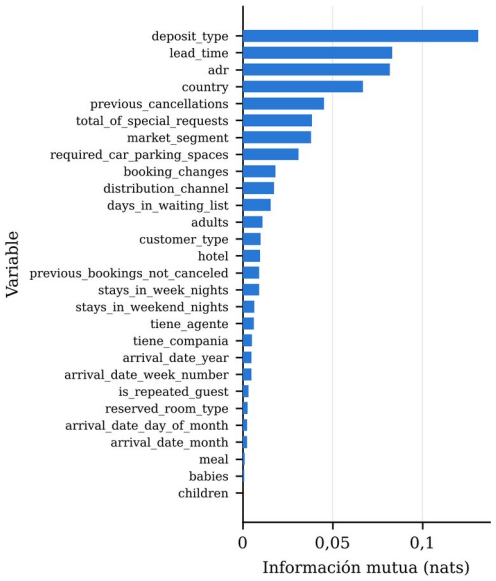


Fig. 4. Información mutua de cada variable de entrada con is_canceled, calculada en el conjunto de entrenamiento.

Se consideraron candidatas a eliminar las nueve variables con información mutua menor a 0,005: arrival_date_year, arrival_date_week_number, is_repeated_guest, reserved_room_type, arrival_date_day_of_month, arrival_date_month, meal, babies y children, que incluyen el par redundante. Al retirarlas, XGBoost pasó de 28 a 19 variables de entrada y su F1 de validación bajó de 0,811 ± 0,011 a 0,790 ± 0,013. Es decir, aunque estas variables aportan poco de forma individual, sí contribuyen en combinación con las demás, posiblemente porque las fechas de llegada complementan a lead_time y a la tarifa.

B. Extracción lineal de características: PCA

PCA se aplicó sobre las 90 variables codificadas. Como criterio se conservó el menor número de componentes que explica al menos el 95 % de la varianza del conjunto de entrenamiento: 25 componentes (Fig. 5a), es decir, una reducción del 72,2 %. Con estas componentes se volvieron a ajustar los hiperparámetros de los dos mejores modelos con la misma validación cruzada. La Fig. 5b muestra que con menos de 20 componentes el F1 de XGBoost cae rápidamente, y que a partir de 25 se acerca al del espacio original sin alcanzarlo.

Con PCA, el F1 de validación de XGBoost bajó de 0,811 a 0,785 ± 0,007 y el de Random Forest de 0,809 a 0,777 ± 0,006, con un AUC-ROC de prueba de 0,924 y 0,918 (Tabla V). Los hiperparámetros óptimos cambiaron en el nuevo espacio: árboles de profundidad 6 en lugar de 9 en XGBoost, y hojas con al menos 5 muestras en Random Forest. Además, la búsqueda de Random Forest tardó casi tres veces más, probablemente porque las componentes son variables continuas, con muchos más puntos de corte posibles que las variables One-Hot.

C. Extracción no lineal de características: UMAP

Por su costo computacional, UMAP se aplicó sobre la muestra de 15.894 reservas. El número de componentes (2, 5, 10 o 20) y los hiperparámetros del clasificador se escogieron por el F1 de validación cruzada, ajustando UMAP en cada pliegue solo con los datos de entrenamiento. Para que la comparación fuera justa, cada modelo se evaluó también en el espacio original con la misma muestra. El mejor resultado se obtuvo con 10 componentes en XGBoost (86,1 % de reducción frente a las 72 variables codificadas de la muestra) y con 20 en Random Forest (72,2 %), pero el F1 de validación fue de solo 0,632 ± 0,015 y 0,633 ± 0,021, respectivamente, frente a 0,777 y 0,775 en el espacio original (Fig. 5c y Tabla V). Una posible explicación es que UMAP construye la proyección sin usar la etiqueta y busca conservar la estructura de vecindad de los datos, dominada aquí por las variables One-Hot, de modo que se pierde parte de la información útil para separar las clases. Además, por ser un método estocástico, sus resultados pueden variar ligeramente entre ejecuciones.

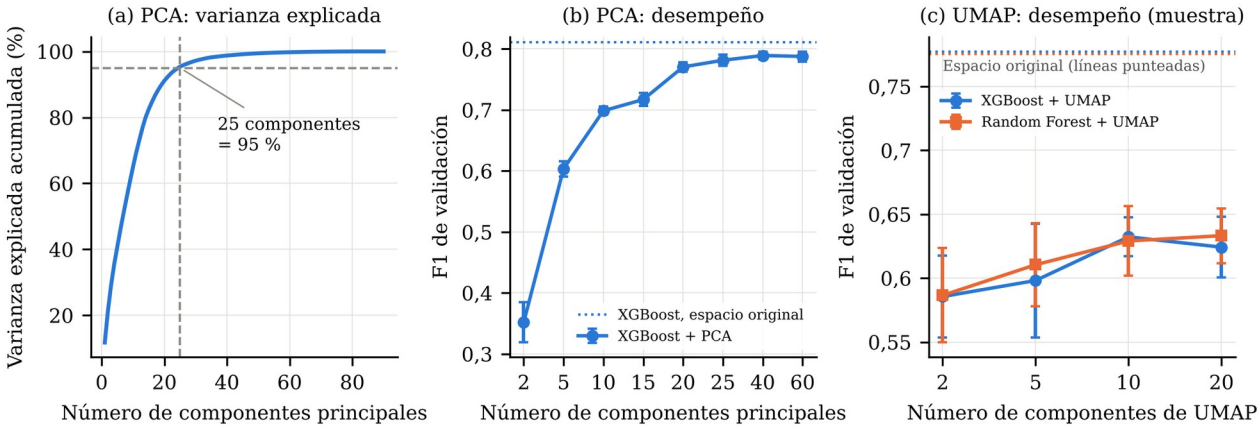


Fig. 5. (a) Varianza explicada acumulada por PCA. (b) F1 de validación de XGBoost según el número de componentes principales. (c) F1 de validación con UMAP según el número de componentes, en la muestra de 15.894 reservas; las líneas punteadas indican el desempeño en el espacio original con la misma muestra.

TABLA V
DESEMPEÑO DE LOS DOS MEJORES MODELOS CON REDUCCIÓN DE DIMENSIÓN

Modelo	**Espacio**	**Datos**	**Dimensión**	**Reducción (%)**	**F1 valid.**	**AUC valid.**	**F1 prueba**	**AUC prueba**
XGBoost	Original	Completos	90	0	0,811 ± 0,011	0,938 ± 0,004	0,820	0,943
XGBoost	PCA	Completos	25	72,2	0,785 ± 0,007	0,921 ± 0,004	0,790	0,924
XGBoost	Original	Muestra	72	0	0,777 ± 0,009	0,914 ± 0,008	0,782	0,919
XGBoost	UMAP	Muestra	10	86,1	0,632 ± 0,015	0,780 ± 0,018	0,622	0,783
Random Forest	Original	Completos	90	0	0,809 ± 0,012	0,936 ± 0,003	0,818	0,941
Random Forest	PCA	Completos	25	72,2	0,777 ± 0,006	0,915 ± 0,005	0,776	0,918
Random Forest	Original	Muestra	72	0	0,775 ± 0,017	0,914 ± 0,010	0,787	0,924
Random Forest	UMAP	Muestra	20	72,2	0,633 ± 0,021	0,787 ± 0,018	0,643	0,797

Validación: media ± IC95 de los cinco pliegues, con nuevo ajuste de hiperparámetros en cada espacio. Muestra: 15.894 reservas del conjunto de entrenamiento.

VI. DISCUSIÓN Y CONCLUSIONES

Los ensambles de árboles fueron los mejores modelos, con un F1 de prueba de 0,820 (XGBoost) y 0,818 (Random Forest) y un AUC-ROC de 0,94, lo que coincide con lo reportado en la literatura [3]–[6]. El AUC-ROC obtenido es muy similar al de Arasadina et al. [5] con Random Forest (0,9394), aun cuando en este trabajo las particiones fueron agrupadas para que reservas idénticas no quedaran en entrenamiento y prueba a la vez. La exactitud superior al 95 % de Adil et al. [3] no es directamente comparable, porque se obtuvo separando los hoteles y balanceando las clases con SMOTE-ENN; nuestros mejores modelos alcanzaron una exactitud de prueba de 0,864 y 0,867. Por otra parte, nuestros resultados superan los de Yang y Miao [4] con modelos de árboles (AUC-ROC de 0,77 a 0,78), quienes usaron una partición temporal y excluyeron country; en nuestro caso, retirar country redujo el AUC-ROC de validación a 0,89, lo que muestra el peso de esta variable y de la metodología de validación en cualquier comparación.

En cuanto a la reducción de dimensión, PCA permitió pasar de 90 a 25 variables (72,2 % menos) con una pérdida de 0,026 a 0,032 en el F1 de validación, mientras que con UMAP la pérdida fue de alrededor de 0,14. El análisis individual mostró además que retirar las nueve variables menos informativas reduce el F1 en 0,021. Por lo tanto, en este problema no se justifica reducir la dimensión: 90 variables no representan un costo alto para los modelos de árboles, y el modelo final recomendado es XGBoost en el espacio original.

Desde el punto de vista práctico, el modelo final detecta cerca del 84 % de las cancelaciones con una precisión del 80 %, lo que permitiría a un hotel concentrar acciones como solicitar un depósito o confirmar la reserva en los casos de mayor riesgo. Entre las limitaciones del trabajo están la posible fuga de información asociada a country, el uso de una muestra para KNN, SVM y UMAP, y que no se evaluó una partición temporal como la de [4] y [6], que reflejaría mejor el uso real del modelo, en el que siempre se predicen reservas futuras con datos del pasado. Estos aspectos, junto con el ajuste del umbral de decisión según los costos del hotel, quedan como trabajo futuro.

REFERENCIAS

[1] N. Antonio, A. de Almeida y L. Nunes, “Hotel booking demand datasets,” *Data in Brief*, vol. 22, pp. 41–49, 2019, doi: 10.1016/j.dib.2018.11.126.

[2] Mojtaba, “Hotel Booking,” Kaggle, 2021. [En línea]. Disponible en: https://www.kaggle.com/datasets/mojtaba142/hotel-booking

[3] M. Adil, M. F. Ansari, A. Alahmadi, J.-Z. Wu y R. K. Chakrabortty, “Solving the problem of class imbalance in the prediction of hotel cancelations: A hybridized machine learning approach,” *Processes*, vol. 9, no. 10, Art. no. 1713, 2021, doi: 10.3390/pr9101713.

[4] D. Yang y X. Miao, “Predicting hotel booking cancellations using tree-based neural network,” *PeerJ Computer Science*, vol. 10, Art. no. e2473, 2024, doi: 10.7717/peerj-cs.2473.

[5] F. Arasadina, S. Briandoko y M. A. Setiawan, “Prediksi pembatalan pemesanan hotel: Random Forest dan XGBoost dengan pipeline leak-free dan ablation study,” *MALCOM: Indonesian Journal of Machine Learning and Computer Science*, vol. 6, no. 3, pp. 1018–1030, 2026, doi: 10.57152/malcom.v6i3.2674.

[6] P. Silvestre, N. Antonio y P. Carrasco, “Navigating uncertainty: enhancing hotel cancellation predictions with adaptive machine learning,” *Information Technology & Tourism*, vol. 28, no. 1, Art. no. 9, 2026, doi: 10.1007/s40558-025-00349-9.